

MARAN PMS — Módulo de Cajas

PROMPT: Turnos Siempre Activos — Autocreación al Cierre

CONTEXTO Y PROBLEMA ACTUAL

El sistema actual requiere que el empleado **abra manualmente un turno** antes de registrar cualquier movimiento de caja. Si se olvida de abrirlo, las ventas quedan sin caja asignada o el sistema falla.

Este prompt modifica la lógica de turnos para que **siempre haya un turno abierto** en cada área. Nunca puede haber un hueco sin turno.

REGLAS DE NEGOCIO

Por área:

- **Recepción:** múltiples turnos por día (mañana / tarde / noche). Cada turno tiene un operador asignado.
- **Restaurant:** un solo turno por día. Se cierra al final del servicio.
- **Spa:** un solo turno por día. Se cierra al final de la jornada.

Al cerrar un turno:

1. El empleado cuenta el efectivo físico
2. Lo entrega a Administración
3. El sistema registra el cierre con el monto contado
4. **Inmediatamente** se crea un nuevo turno con saldo inicial \$0
5. En Recepción: se pide el nombre del operador que toma el nuevo turno
6. En Restaurant/Spa: el nuevo turno queda abierto sin operador asignado hasta que

alguien inicie sesión

Fallback automático:

- Si por algún motivo no existe un turno abierto al registrar un movimiento, el sistema **crea uno automáticamente** en segundo plano antes de registrar el movimiento
 - Este turno “fantasma” queda marcado con `autoCreado = true` para que administración lo vea y lo gestione
-

PARTE 1 — CAMBIOS EN LA BASE DE DATOS

Modificar tabla de turnos existente (`cash_shifts` o equivalente)

Agregar los siguientes campos:

```
// Agregar a la tabla de turnos existente:

autoCreado: boolean("auto_creado").default(false),
// true si el turno fue creado automáticamente por el sistema (sin acción del emp

turnoAnteriorId: integer("turno_anterior_id"),
// FK al turno que se cerró y originó este (para trazabilidad)

operador: text("operador"),
// Nombre del empleado que toma el turno
// Puede ser null si el turno fue autocreado y aún no lo tomó nadie

// El saldo inicial SIEMPRE es 0 (el efectivo se retira al cerrar)
// El campo saldoInicial ya debería existir, confirmar que su default es 0
```

PARTE 2 — LÓGICA DE CIERRE Y AUTOCREACIÓN

`server/cashShifts.ts` — Modificar función de cierre

```
export async function cerrarTurno(
  turnoId: number,
  efectivoContado: number,
  operadorSiguiente: string | null,
  enviarAAministracion: boolean,
  operadorActual: string
```

```

): Promise<{ turnoCerrado: CashShift; turnoNuevo: CashShift }> {

  // 1. Cerrar el turno actual
  const [turnoCerrado] = await db
    .update(cashShifts)
    .set({
      estado: "cerrado",
      fechaCierre: new Date(),
      efectivoContado,
      diferencia: efectivoContado - calculatedBalance, // saldo sistema - contado
      operadorCierre: operadorActual,
    })
    .where(eq(cashShifts.id, turnoId))
    .returning();

  // 2. Si el área es Recepción y se marcó "enviar a administración",
  //     crear automáticamente el movimiento en la Caja de Administración
  if (enviarAAAdministracion && efectivoContado > 0) {
    await db.insert(adminCashMovements).values({
      fecha: format(new Date(), "yyyy-MM-dd"),
      hora: format(new Date(), "HH:mm"),
      tipo: `ingreso_${turnoCerrado.area}`, // "ingreso_recepcion" | "ingreso_res
      concepto: `Efectivo ${capitalize(turnoCerrado.area)} - Turno #${turnoId} ($
      importe: efectivoContado.toString(),
      signo: "+",
      areaOrigen: turnoCerrado.area,
      cierreOrigenId: turnoId,
      operador: operadorActual,
    });
  }

  // 3. Crear inmediatamente el nuevo turno con saldo $0
  const [turnoNuevo] = await db.insert(cashShifts).values({
    area: turnoCerrado.area,
    estado: "abierto",
    fechaApertura: new Date(),
    saldoInicial: 0, // siempre arranca en $0
    operador: operadorSiguiente ?? null,
    autoCreado: false, // este lo creó el sistema tras el cierre, pero hay
    turnoAnteriorId: turnoId,
  }).returning();

  return { turnoCerrado, turnoNuevo };
}

```

Función de fallback automático

```

// Llamar esta función ANTES de registrar cualquier movimiento de caja
export async function getOrCreateActiveTurno(area: string): Promise<CashShift> {
  // Buscar turno abierto para el área
  const turnoActivo = await db
    .select()
    .from(cashShifts)
    .where(
      and(
        eq(cashShifts.area, area),
        eq(cashShifts.estado, "abierto")
      )
    )
    .limit(1);

  if (turnoActivo.length > 0) {
    return turnoActivo[0];
  }

  // No hay turno abierto → crear uno automáticamente
  console.warn(`[CashShift] No había turno abierto para ${area}. Autocreando.`);

  const [turnoNuevo] = await db.insert(cashShifts).values({
    area,
    estado: "abierto",
    fechaApertura: new Date(),
    saldoInicial: 0,
    operador: null,
    autoCreado: true, // ← marcado para que administración lo vea
  }).returning();

  return turnoNuevo;
}

```

Integrar el fallback en TODOS los endpoints que registran movimientos

```

// Ejemplo: al registrar un pago en el folio
app.post("/api/folios/:id/payments", async (req, res) => {
  const area = "recepcion";

  // ← AGREGAR ESTA LÍNEA antes de cualquier insert de movimiento
  const turno = await getOrCreateActiveTurno(area);

  await db.insert(folioPayments).values({
    ...paymentData,
    cashShiftId: turno.id, // siempre queda vinculado a un turno
  });
});

```

```
});  
});  
  
// Lo mismo para:  
// - Pagos de restaurant  
// - Pagos de Spa  
// - Cualquier otro movimiento que impacte caja
```

PARTE 3 — FRONTEND: FLUJO DE CIERRE DE TURNO

Modificar el `CierreTurnoDialog` existente

El dialog de cierre debe reflejar el nuevo flujo en dos pasos:

Paso 1 — Conteo de efectivo

Cierre de Turno — Recepción		Turno #142
Apertura: 08:00hs Operador: María González		
RESUMEN DEL TURNO		
Efectivo recibido:		\$45.200,00
Tarjetas:		\$32.800,00
Transferencias:		\$18.600,00
Total cobrado:		\$96.600,00
CONTEO DE EFECTIVO FÍSICO		
Billetes \$1.000:	[__] × \$1.000	= \$ ____
Billetes \$500:	[__] × \$500	= \$ ____
Billetes \$200:	[__] × \$200	= \$ ____
Billetes \$100:	[__] × \$100	= \$ ____
Billetes \$50:	[__] × \$50	= \$ ____
Monedas:	[_____]	
Total contado:	\$45.200,00	
Diferencia:	\$0,00	✓
✓ Enviar efectivo a Caja Administración (se registrará automáticamente el ingreso)		
[Cancelar] [Siguiente →]		

Paso 2 — Apertura del siguiente turno (solo Recepción)

Cierre de Turno — Recepción	Turno #142
☒ Turno cerrado correctamente	
PRÓXIMO TURNO	
El sistema abrirá automáticamente el Turno #143 con saldo inicial \$0,00	
¿Quién toma el siguiente turno?	
Operador:	[_____] (dejar vacío si no hay relevo inmediato)
Hora de inicio del nuevo turno: 16:00hs (ahora)	
[Cancelar] [Confirmar y Cerrar]	

Para **Restaurant y Spa** no se muestra el Paso 2 (el turno nuevo se crea solo, sin asignar operador).

Confirmación final

- ☒ Turno #142 cerrado
- ☒ \$45.200,00 registrados en Caja Administración
- ☒ Turno #143 abierto — Operador: Carlos Rodríguez

PARTE 4 — INDICADOR VISUAL DE TURNO ACTIVO

En el header/navbar del sistema

Mostrar siempre el estado del turno activo del área del usuario logueado:

[ Turno #143 · Carlos R. · desde 16:02hs]

-  Verde: turno normal activo
-  Amarillo: turno autocreado sin operador asignado
-  Rojo: no debería aparecer nunca (pero si ocurre, indica error crítico)

Al hacer clic, abre un mini panel:

```
Turno #143 — Recepción
Operador: Carlos Rodríguez
Apertura: 16:02hs
Efectivo en caja: $0,00
Tarjetas: $8.400,00
[Cerrar turno] [Tomar turno]
```

“Tomar turno” aparece cuando el turno fue autocreado o no tiene operador asignado. Permite que el empleado ingrese su nombre sin abrir un turno nuevo.

PARTE 5 — ALERTA DE TURNOS AUTOCREADOS

En el panel de Administración

Si el sistema creó turnos automáticamente (por fallback), mostrar una alerta:

```
⚠ Atención: 2 turnos fueron autocreados por el sistema hoy.
  Esto indica que había ventas sin turno abierto.
  [Ver detalle]
```

El detalle muestra qué área, a qué hora ocurrió, y qué movimientos quedaron en ese turno.

PARTE 6 — CASOS BORDE

Inicio del sistema / primera vez

Al arrancar el sistema por primera vez (o luego de una migración), ejecutar:

```
// server/init.ts
export async function initializeCashShifts() {
  const areas = ["recepcion", "restaurant", "spa"];

  for (const area of areas) {
    const turnoActivo = await db
      .select()
      .from(cashShifts)
      .where(and(eq(cashShifts.area, area), eq(cashShifts.estado, "abierto")))
```

```

        .limit(1);

    if (turnoActivo.length === 0) {
        await db.insert(cashShifts).values({
            area,
            estado: "abierto",
            fechaApertura: new Date(),
            saldoInicial: 0,
            autoCreado: true,
        });
        console.log(`[Init] Turno inicial creado para área: ${area}`);
    }
}

// Llamar en el arranque del servidor:
// server/index.ts → await initializeCashShifts();

```

Múltiples turnos abiertos (situación anómala)

Si por algún bug hubiera más de un turno abierto en la misma área,

`getOrCreateActiveTurno` debe tomar el más reciente y logear una advertencia. No debe fallar.

```

// Si hay más de uno, tomar el más reciente
.orderBy(desc(cashShifts.fechaApertura))
.limit(1)

```

NOTAS

1. **Retrocompatibilidad:** los turnos existentes en la base de datos no se modifican. El campo `autoCreado` es `false` por defecto, así que los turnos históricos quedan limpios.
2. **Restaurant y Spa de un turno por día:** si el sistema detecta que ya hay un turno cerrado para esa área en el día de hoy y se intenta crear otro, verificar que el nuevo turno tenga `fechaApertura` en el mismo día. Si ya hay dos cerrados en el día, el tercero se crea igual (puede pasar si hubo un error y se reabrió).
3. **El checkbox “Enviar efectivo a Administración”** en el cierre viene marcado por defecto en `true`. El empleado puede desmarcarlo si por algún motivo el efectivo no se entrega en ese momento (ej: lo retira el gerente después). En ese caso, el

movimiento en Caja Admin se puede registrar manualmente desde el módulo de Administración.

4. **Trazabilidad completa:** cada movimiento de caja tiene `cashShiftId` , cada turno tiene `turnoAnteriorId` . Se puede reconstruir toda la cadena de turnos desde el inicio del sistema.